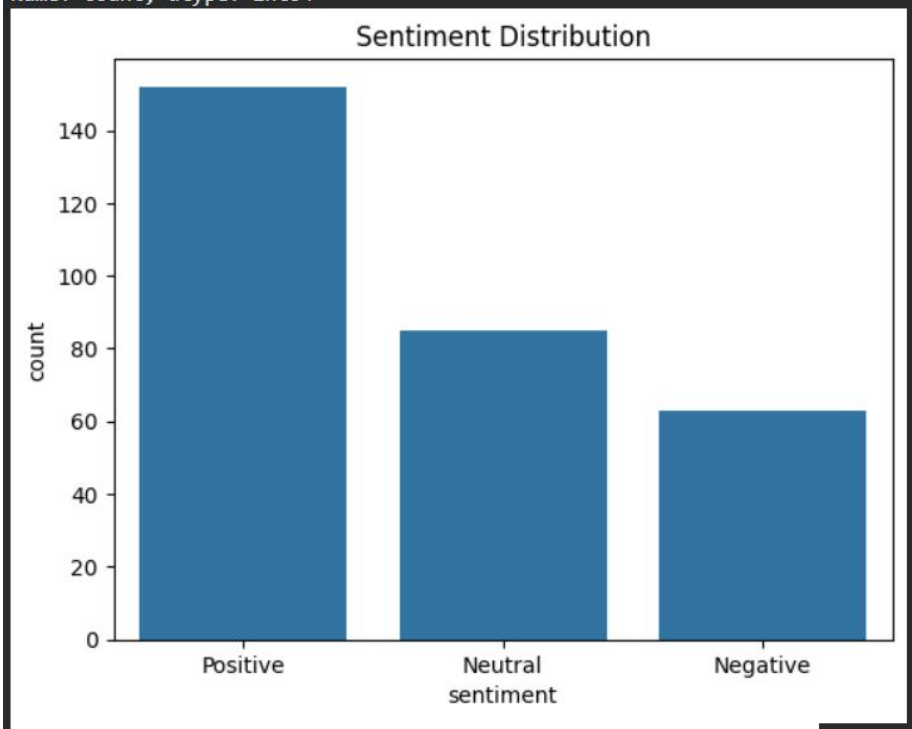
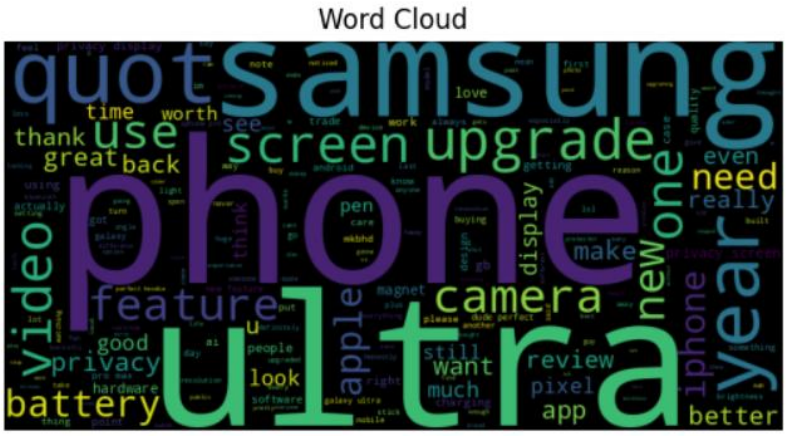
```
from googleapiclient.discovery import build
import pandas as pd
import re
import nltk
import matplotlib.pyplot as plt
import seaborn as sns
from textblob import TextBlob
from nltk.corpus import stopwords
from wordcloud import WordCloud
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from urllib.parse import urlparse, parse_qs
from collections import Counter
nltk.download('stopwords')
API_KEY = "AIzaSyClEBK38SAUlWFfTWjgR5WG-tSKTdsWAYs"

YOUTUBE_URL = "https://youtu.be/ObqHKHoSE5Y?si=TcKos-KOeLyLNvOK"
# Replace with your YouTube video link

def extract_video_id(url):
    parsed_url = urlparse(url)

    if parsed_url.hostname == 'youtu.be':
        return parsed_url.path[1:]

    if parsed_url.hostname in ('www.youtube.com', 'youtube.com'):
        if parsed_url.path == '/watch':
            return parse_qs(parsed_url.query)['v'][0]
        elif parsed_url.path.startswith('/shorts/'):
            return parsed_url.path.split('/')[2]

    raise ValueError("Invalid YouTube URL")

VIDEO_ID = extract_video_id(YOUTUBE_URL)
print(f"Video ID: {VIDEO_ID}")
def get_youtube_comments(video_id, api_key, max_comments=300):
    youtube = build('youtube', 'v3', developerKey=api_key)

    comments = []
    next_page_token = None

    while len(comments) < max_comments:
        request = youtube.commentThreads().list(
            part="snippet",
            videoId=video_id,
            maxResults=100,
            pageToken=next_page_token
        )
        response = request.execute()

        for item in response['items']:
            comment = item['snippet']['topLevelComment']['snippet']['textDisplay']
```

```
            comments.append(comment)

        next_page_token = response.get('nextPageToken')

        if not next_page_token:
            break

    return pd.DataFrame(comments, columns=['comment'])

print("Fetching comments...")
df = get_youtube_comments(VIDEO_ID, API_KEY)
print(f"Total Comments: {len(df)}")

stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = text.lower()
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

df['cleaned'] = df['comment'].apply(clean_text)
def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
        return "Positive"
    elif polarity < 0:
        return "Negative"
    else:
        return "Neutral"

df['sentiment'] = df['cleaned'].apply(get_sentiment)

print("\nSentiment Counts:")
print(df['sentiment'].value_counts())
plt.figure()
sns.countplot(x='sentiment', data=df)
plt.title("Sentiment Distribution")
plt.show()
text = " ".join(df['cleaned'])

wordcloud = WordCloud(width=800, height=400).generate(text)

plt.figure()
plt.imshow(wordcloud)
plt.axis('off')
plt.title("Word Cloud")
plt.show()
vectorizer = CountVectorizer(max_df=0.9, min_df=2, stop_words='english')
X = vectorizer.fit_transform(df['cleaned'])
```

```
lda = LatentDirichletAllocation(n_components=3, random_state=42)
lda.fit(X)

words = vectorizer.get_feature_names_out()

print("\nTopics:")
for i, topic in enumerate(lda.components_):
    print(f"\nTopic {i+1}:")
    print([words[j] for j in topic.argsort()[-10:]])

negative_comments = df[df['sentiment'] == 'Negative']

print("\nSample Negative Comments:")
print(negative_comments['comment'].head(10))

all_words = " ".join(negative_comments['cleaned']).split()
common_words = Counter(all_words).most_common(10)

print("\nTop Complaint Keywords:")
print(common_words)

print("\n===== SMART PRODUCT IMPROVEMENT SUGGESTIONS =====\n")

def generate_suggestions(negative_df):
    suggestions = []

    text_data = " ".join(negative_df['cleaned'])

    # Rule-based intelligence
    if any(word in text_data for word in ["slow", "lag", "delay"]):
        suggestions.append("⚡ Improve performance and reduce lag issues")

    if any(word in text_data for word in ["price", "expensive", "cost"]):
        suggestions.append("💰 Consider reducing price or adding discounts")

    if any(word in text_data for word in ["quality", "bad", "poor"]):
        suggestions.append("🔧 Improve product/service quality")

    if any(word in text_data for word in ["support", "service", "help"]):
        suggestions.append("📞 Enhance customer support experience")

    if any(word in text_data for word in ["bug", "error", "issue"]):
        suggestions.append("🐞 Fix bugs and improve system stability")

    if any(word in text_data for word in ["ads", "advertisement"]):
        suggestions.append("📺 Reduce ads or improve ad experience")

    if any(word in text_data for word in ["update", "new version"]):
        suggestions.append("🔄 Improve updates and add requested features")
```

```
sentiment
Positive    152
Neutral      85
Negative     63
Name: count, dtype: int64
```




```
Topics:

Topic 1:
['battery', 'phones', 'feature', 'ultra', 'samsung', 'phone', 'new', 'screen', 'privacy', 'quot']

Topic 2:
['year', 'make', 'samsung', 'privacy', 'display', 'review', 'like', 'better', 'ultra', 'phone']

Topic 3:
['pro', 'max', 'like', 'iphone', 'video', 'apple', 'upgrade', 'phone', 'samsung', 'ultra']

Sample Negative Comments:
4     So if you don't care about privacy, you basica...
6     considering trading up from the s24 ultra, but...
9     You can play Cyberpunk 2077 and Red Dead Redem...
11    He sounded bias lately 🤣 unlike from when he j...
13    I have the base s26 but I cant imagine how muc...
22    That camera bump is completely unnecessary.. p...
25    We don&#39;t need a magnet on the phones. Such...
31    <a href="https://www.youtube.com/watch?v=nfHRM...
39    <a href="https://www.youtube.com/watch?v=nfHRM...
47    Are all the stupid AI slop options able to be ...
Name: comment, dtype: object

Top Common Keywords:
[('ultra', 30), ('phone', 30), ('samsung', 20), ('get', 11), ('apple', 11), ('like', 10), ('screen', 9), ('year', 9), ('back', 9), ('camera', 8)]

===== SMART PRODUCT IMPROVEMENT SUGGESTIONS =====

1. ⚡ Improve performance and reduce lag issues
2. 💰 Consider reducing price or adding discounts
3. 🔧 Improve product/service quality
4. 📞 Enhance customer support experience
5. 🐞 Fix bugs and improve system stability
```